

FIFO

MEMORIA DESCRIPTIVA

RotaStock — Control de Rotación de Inventario

*Sistema de escritorio portable para valoración FIFO (PEPS),
semaforización de rotación y análisis de cobertura de inventario*

AUTOR : Pedro Giovanni Ricra Figueroa

Pucallpa – Perú

2026

ÍNDICE

1. Introducción.....	3
2. Infraestructura Tecnológica y Arquitectura de Software	4
2.1 Stack Tecnológico de Desarrollo	4
2.2 Arquitectura y Estructura de Directorios	4
2.3 Persistencia portable y copias de seguridad.....	5
3. Objetivos del Sistema.....	5
3.1 Objetivo General	5
3.2 Objetivos Específicos	6
4. Requisitos de Infraestructura del Sistema	6
5. Arquitectura Global de la Interfaz	7
5.1 Cabecera de control global	7
5.2 Barra lateral de navegación.....	7
6. Especificación Detallada de los Módulos del Sistema.....	8
6.1 Módulo 01: Rotación de Inventario (Dashboard)	8
6.2 Módulo 02: Productos (CRUD)	9
6.3 Módulo 03: Movimientos (compras y ventas)	9
6.4 Módulo 04: Configuración de umbrales	10
6.5 Módulo 05: Exportar Datos (Excel)	10
6.6 Sección de Cierre: Auditoría Técnica.....	11
7. Modelado Matemático-Logístico y Algoritmos de Negocio.....	11
7.1 Replay cronológico y valoración FIFO estricta	11
7.2 Indicadores de rotación, cobertura y días de inventario	12
7.3 Semaforización por días sin movimiento	13
7.4 Período de análisis y agregación del dashboard.....	14
8. Directrices de Operación, Recomendaciones y Mantenimiento.....	14
9. Conclusiones	15
10. Referencias Técnicas.....	16
Anexo A. Instancia de demostración (equipos de red)	17
Anexo B. Esquema JSON de persistencia.....	18

1. Introducción

El control de rotación de inventario es una de las palancas más sensibles de la logística de almacén: un SKU que no se mueve inmoviliza capital, ocupa espacio y se deteriora; un SKU que rota demasiado rápido sin cobertura suficiente genera quiebres de stock. Medir esa dinámica exige, además, una política de costeo coherente: si el costo de lo vendido no refleja el orden real de ingreso de los lotes, los indicadores de rotación, cobertura y ganancia bruta se vuelven incomparables entre períodos.

RotaStock es una aplicación de escritorio portable (Electron) para el control de rotación de inventario con valoración FIFO estricta, también conocida como PEPS (primero en entrar, primero en salir). El operador registra productos y movimientos de compra y venta; el motor reconstruye, desde cero y en orden cronológico, los lotes vivos, el stock, el COGS, los ingresos y la ganancia bruta. Sobre ese estado derivado calcula, para el período elegido en la cabecera (de una semana a todo el historial), los días de inventario, la rotación en veces, los días de cobertura y los días sin movimiento, y semaforiza cada SKU en Rotación Sana, Lenta, Muy Lenta o Inventario Dormido.

A diferencia de una hoja de cálculo, el sistema no almacena indicadores precalculados como fuente de verdad. Cualquier alta, edición o baja de un movimiento dispara un replay completo: el stock, el COGS de cada venta y las alertas se recalculan a partir de la bitácora. Esa decisión de diseño elimina la deriva entre «lo que muestra el dashboard» y «lo que ocurrió en el almacén», y hace auditable cada cifra. El presente documento describe la infraestructura, cada módulo de la interfaz, el modelo FIFO y las directrices de uso, con el mismo rigor de las demás herramientas del curso de Logística.

2. Infraestructura Tecnológica y Arquitectura de Software

RotaStock está diseñado como una aplicación de escritorio offline, empaquetable en un único ejecutable portable de Windows (RotaStock.exe). El frontend es una SPA de HTML5/CSS/JavaScript Vanilla que también puede abrirse en el navegador, usando localStorage como respaldo cuando no hay proceso Electron.

2.1 Stack Tecnológico de Desarrollo

- Contenedor de escritorio: Electron 30, con contextIsolation activo y nodeIntegration desactivado. El proceso principal (main.js) no expone el sistema de archivos al renderer: solo un puente IPC (db:load / db:save).
- Puente seguro: preload.js publica window.rotastock mediante contextBridge (isElectron, loadDB, saveDB).
- Presentación: HTML5 + CSS3 con temas claro y oscuro (data-theme), persistido en localStorage como rotastock_theme, aplicado antes del primer pintado para evitar parpadeo.
- Lógica de negocio: JavaScript Vanilla (frontend/script.js). El objeto Engine encapsula el replay FIFO, las métricas por producto y el dashboard. No hay frameworks ni bundlers.
- Gráficos: Chart.js 4 (chart.umd.min.js, vendorizado). Barras de Ingresos vs. COGS y dona de distribución de estados.
- Exportación: SheetJS / xlsx.full.min.js, vendorizado, para generar un libro de tres hojas sin servidor.
- Empaquetado: electron-builder en modo portable (npm run build → dist/RotaStock.exe).

Capa	Tecnología	Función
Escritorio	Electron 30 + electron-builder	Ventana nativa y .exe portable
IPC	preload.js + contextBridge	Carga y guardado de la base JSON
Presentación	HTML5 + CSS3 (claro/oscuro)	SPA de cinco vistas
Motor	JavaScript Vanilla (Engine)	Replay FIFO y métricas de rotación
Visualización	Chart.js 4 (UMD local)	Barras y dona del dashboard
Exportación	SheetJS (xlsx)	Reporte Excel de tres hojas
Persistencia	inventario.json + backups/	Fuente de verdad local, offline

Tabla 1. Stack tecnológico de RotaStock.

2.2 Arquitectura y Estructura de Directorios

El proyecto se organiza en un árbol plano, pensado para copiarse a un USB y ejecutarse sin instalador:

- main.js — proceso principal: rutas portables, carga/guardado JSON, backups automáticos, creación de la ventana 1 360 × 860 (mínimo 1 024 × 640).
- preload.js — puente IPC de dos métodos.
- frontend/index.html, style.css, script.js — interfaz y motor.
- frontend/assets/vendor/ — Chart.js y SheetJS, sin CDN.
- datos/inventario.json — base de datos local (productos, movimientos, umbrales).

- backups/ — copias automáticas inventario-<ISO>.json, a lo sumo una por minuto.

En desarrollo, datos/ y backups/ viven en la raíz del proyecto. En el .exe empaquetado, el frontend se sirve desde el app.asar (solo lectura) y los JSON se escriben junto al ejecutable, no dentro del asar. `getBasePath()` distingue ambos modos con `app.isPackaged`.

2.3 Persistencia portable y copias de seguridad

El esquema mínimo de la base es { meta, config, productos, movimientos }. Si el archivo no existe, se crea con umbrales por defecto (sana = 7, lenta = 15, muyLenta = 30) y arreglos vacíos. Un fallo de lectura cae a `defaultDB()` para no bloquear el arranque. Cada `persistAndRender()` escribe `inventario.json` y, si pasaron más de 60 s desde la última copia, un backup fechado. En el navegador, `Store.save` escribe en `localStorage` (clave `rotastock_db`) y `Store.load` intenta primero esa clave, luego un fetch de `datos/inventario.json` como semilla.

3. Objetivos del Sistema

3.1 Objetivo General

Desarrollar una herramienta de escritorio portable, 100 % offline, que permita registrar el inventario y sus movimientos, valorarlos con FIFO estricto y diagnosticar la rotación de cada producto —cobertura, días sin movimiento y semáforo de estados— en un período de análisis configurable.

3.2 Objetivos Específicos

1. Mantener un maestro de productos con alta, edición y baja, incluyendo stock inicial opcional como compra FIFO.
2. Registrar compras y ventas con autocompletado de producto, cantidades enteras y validación de stock disponible en ventas.
3. Reconstruir lotes, stock y COGS mediante replay cronológico FIFO cada vez que cambia la bitácora.
4. Calcular, para un período (7, 15, 30, 90, 180, 365 días o todo el historial), rotación, cobertura, días de inventario, COGS, ingresos y ganancia bruta.
5. Semaforizar cada SKU según días sin movimiento y umbrales configurables (Sana / Lenta / Muy Lenta / Dormido).
6. Alertar en el dashboard cuando existan productos en Inventario Dormido.
7. Exportar un libro Excel de tres hojas (Resumen, Productos, Movimientos) acotado al período de la cabecera.

4. Requisitos de Infraestructura del Sistema

Componente técnico	Requisito mínimo del sistema
Procesador / CPU	Intel Core i3 o equivalente.
Memoria RAM	4 GB (8 GB recomendados para el empaquetado Electron).
Pantalla	1 024 × 640 px mínimo; diseño pensado para 1 360 × 860.
Sistema operativo	Windows 10 o superior para el .exe portable; el frontend también corre en navegador.
Runtime de desarrollo	Node.js LTS (npm install, npm start, npm run build).
Conectividad	Ninguna en operación. Internet solo para la primera instalación de dependencias.
Almacenamiento	Carpeta escribible junto al ejecutable (datos/ y backups/).

Tabla 2. Requisitos mínimos de infraestructura.

5. Arquitectura Global de la Interfaz

La interfaz es una SPA de dos columnas sobre un fondo industrial oscuro (#161816) con modo claro opcional. Cinco vistas se intercambian sin recargar: Rotación, Productos, Movimientos, Configuración y Exportar Datos. Un modal genérico cubre altas, ediciones y confirmaciones de baja; un toast confirma las operaciones.

5.1 Cabecera de control global

La barra superior concentra los controles que afectan a todas las vistas: botón de menú (oculta o muestra el sidebar), marca RotaStock, selector de período (1 semana, 15 días, 1 mes por defecto, 3 meses, 6 meses, 1 año, Siempre), conmutador de tema claro/oscuro y botón Actualizar, que reejecuta el render y notifica «Datos actualizados». El período no es un filtro cosmético: redefine el rango [start, end] con el que Engine calcula COGS, ingresos, rotación y cobertura. El Excel exportado respeta exactamente ese rango.

5.2 Barra lateral de navegación

Cinco ítems con iconos SVG de línea: Rotación, Productos, Movimientos, Configuración y Exportar Datos. El pie del sidebar exhibe el método de costeo fijo: FIFO (PEPS). No hay selector de LIFO ni de costo promedio: la política es una constante de diseño (config.metodoCosteo = 'FIFO'), no un parámetro de usuario, para no mezclar series históricas.

6. Especificación Detallada de los Módulos del Sistema

6.1 Módulo 01: Rotación de Inventario (Dashboard)

Vista de arranque. Un banner de alerta se muestra cuando hay al menos un producto en estado Dormido y puede cerrarse con × sin borrar el KPI. Cinco tarjetas encabezan el diagnóstico: Días de Inventario, COGS del período, Ingresos Totales, Ganancia Bruta y Alertas de Rotación. Debajo, un gráfico de barras compara ingresos y COGS de los ocho productos de mayor ingreso, y una dona cuenta SKU por estado (Sana, Lenta, Muy Lenta, Dormido).

La tabla «Detalle por producto» lista inventario promedio (unidades), rotación (veces), días de cobertura, días sin movimiento y el píldora de estado. Soporta búsqueda, orden por columna (clic en el encabezado) y paginación de 8 filas. El pie de página del período se escribe en locale es-PE (dd/mm/aaaa → dd/mm/aaaa).

6.2 Módulo 02: Productos (CRUD)

Rejilla de tarjetas. Cada tarjeta muestra nombre, estado, stock actual, inventario promedio, COGS, ingresos, ganancia, rotación, cobertura y días sin movimiento, más acciones Editar y Eliminar. La barra permite buscar por nombre, filtrar por estado y ordenar por siete campos en sentido ascendente o descendente.

Alta: nombre obligatorio; stock inicial y costo total del lote son opcionales, pero si hay stock > 0 el costo total es obligatorio. El sistema no guarda un campo «stock» en el maestro: inserta el producto y, si aplica, una compra con costo unitario = costoTotal / stock (cuatro decimales). Edición: solo cambia el nombre. Baja: confirma cuántos movimientos se eliminarán en cascada y recalcula todos los indicadores.

6.3 Módulo 03: Movimientos (compras y ventas)

Tabla paginada del historial completo (no filtrada por período: el período afecta al análisis, no a la bitácora). Columnas: fecha y hora (es-PE), producto, tipo (Compra / Venta), cantidad entera, costo o precio unitario, total = cantidad × unitario, y acciones. Filtros: búsqueda por nombre y tipo (todas, solo compras, solo ventas). Orden por encabezado.

El formulario de registro exige elegir el producto de una lista de autocompletado (el id oculto se vacía si el texto cambia, para no grabar un SKU ambiguo). El rótulo del importe cambia según el tipo: «Costo de compra unitario» o «Precio de venta unitario». En ventas se valida el stock FIFO actual; si se edita una venta, se reintegra temporalmente su cantidad al

disponible. Una venta que exceda el stock se rechaza con el mensaje «Stock insuficiente: disponible N unidad(es)». Toda alta, edición o baja dispara `persistAndRender()`: el COGS de cada venta se reescribe en `m._cogs` y el ingreso en `m._ingreso`.

6.4 Módulo 04: Configuración de umbrales

Formulario de tres enteros: Rotación Sana — hasta, Rotación Lenta — hasta, Rotación Muy Lenta — hasta. Lo que supera «Muy Lenta» cae automáticamente en Inventario Dormido; no hay un cuarto umbral editable. La validación exige $Sana \leq Lenta \leq Muy\ Lenta$. «Restablecer por defecto» vuelve a 7 / 15 / 30 días. Los umbrales no alteran el FIFO: solo la semaforización y el KPI de alertas.

6.5 Módulo 05: Exportar Datos (Excel)

Un panel recuerda el período activo y describe las tres hojas. El botón Generar y descargar Excel llama a `XLSX.writeFile` con el nombre `RotaStock_<periodo>_<aaaa-mm-dd>.xlsx`.

Hoja	Contenido
Resumen	Período, fechas, método FIFO, días de inventario, COGS, ingresos, ganancia, alertas.
Productos	Stock, inv. promedio, rotación, cobertura, días sin mov., estado.
Movimientos	Historial del período: cantidad, unitario, COGS, ingreso, ganancia (0 en compras).

Tabla 3. Estructura del libro Excel exportado.

6.6 Sección de Cierre: Auditoría Técnica

- Fuente de verdad: solo productos y movimientos. Stock, COGS e indicadores son derivados del replay.
- Cantidades enteras: `parseInt` en compras, ventas y stock inicial; no se admiten fracciones de unidad.
- Escape HTML en nombres y toasts para evitar inyección en el DOM.
- Movimientos huérfanos (`productoId` inexistente) se ignoran en el replay.
- Faltante FIFO: si una venta histórica supera los lotes (p. ej. tras borrar una compra), el residual se valora al último costo conocido (`lastCost`), para no romper el recálculo.
- El costeo es FIFO y no se puede cambiar desde la interfaz: evita series mezcladas LIFO / promedio.

7. Modelado Matemático-Logístico y Algoritmos de Negocio

El núcleo no es un solver numérico: es un reloj de almacén. Se ordenan los movimientos por fecha (empate por id), se recorren uno a uno y se mantiene, por producto,

una cola de lotes { remaining, costo, fecha, movId }. Esa cola es la implementación literal de PEPS.

7.1 Replay cronológico y valoración FIFO estricta

Compra de q unidades a costo unitario c en el instante t :

$$lotes \leftarrow lotes \cup \{ remaining = q, costo = c, fecha = t \}$$

$$COGS_{compra} = q \cdot c \quad (\text{costo de adquisición, no costo de ventas})$$

$$Ingreso_{compra} = 0$$

Venta de q unidades a precio p . Se consumen lotes desde el más antiguo:

$$COGS_{venta} = \sum take_k \cdot costo_k$$

$$Ingreso_{venta} = q \cdot p$$

$$Ganancia_{venta} = Ingreso_{venta} - COGS_{venta}$$

$take_k = \min(remaining_k, pendiente)$. El lote se retira de la cola cuando $remaining \leq 10^{-9}$.

Tras cada movimiento se actualiza el stock como suma de remaining y el valor de inventario como $\sum remaining \cdot costo$, y se guarda un snapshot { fecha, qty, val } para poder interpolar el inventario en cualquier instante del período (último snapshot con fecha $\leq t$).

7.2 Indicadores de rotación, cobertura y días de inventario

Sea $[t_0, t_1]$ el período y $\Delta = \max(1, \text{días entre } t_0 \text{ y } t_1)$. Para cada producto se acumulan las ventas con fecha en el intervalo:

$$COGS = \sum COGS_{venta} \quad Ingresos = \sum q \cdot p \quad Ganancia = Ingresos - COGS$$

$$I_0 = \text{valor de inventario en } t_0 \quad I_1 = \text{valor de inventario en } t_1$$

$$Inventario \text{ promedio} = (I_0 + I_1) / 2$$

$$Rotación \text{ (veces)} = COGS / Inventario \text{ promedio} \quad (0 \text{ si el promedio es } 0)$$

$$Cobertura \text{ (días)} = I_1 / (COGS / \Delta) \quad (\text{indefinida si } COGS = 0)$$

A nivel de almacén, los días de inventario del KPI principal usan el valor actual de todos los SKU (no el promedio) sobre el ritmo de COGS del período:

$$Días \text{ de inventario} = (\sum valor_inventario_actual) / (COGS_total / \Delta)$$

Es la pregunta operativa «¿cuántos días aguanta el stock de hoy al ritmo de ventas de este período?».

7.3 Semaforización por días sin movimiento

$$\text{días sin mov.} = \max(0, \lfloor (t_1 - \text{última_fecha_movimiento}) / 86\,400\,000 \rfloor)$$

Con umbrales (s, ℓ, m) estrictamente no decrecientes:

- Sana si $\text{días} \leq s$.
- Lenta si $s < \text{días} \leq \ell$.
- Muy lenta si $\ell < \text{días} \leq m$.
- Dormido si $\text{días} > m$.
- Sin datos si el producto nunca tuvo movimientos.

Valores de fábrica: $s = 7$, $\ell = 15$, $m = 30$. El KPI «Alertas de Rotación» cuenta únicamente los Dormido. La semaforización no usa la rotación en veces: un producto puede tener rotación alta en el período y, aun así, estar Dormido si lleva más de m días sin un movimiento nuevo. Esa doble lectura (velocidad histórica vs. silencio reciente) es deliberada.

7.4 Período de análisis y agregación del dashboard

El extremo derecho del rango es siempre ahora (`Date.now()`). El izquierdo es ahora – $N \cdot 86400000$ ms, o la fecha del primer movimiento global si el operador elige «Siempre». Las compras y ventas fuera del rango no entran a COGS ni a ingresos del período, pero sí a la cola FIFO: el stock actual y el valor de inventario se reconstruyen con toda la historia. Por eso un período de 7 días puede mostrar un inventario alto con COGS bajo (poca venta reciente) y cobertura grande: el motor no recorta los lotes al período, recorta los flujos.

8. Directrices de Operación, Recomendaciones y Mantenimiento Técnico

- Registrar primero el producto y después sus movimientos. Una venta sin stock se rechaza; no se permiten saldos negativos.
- El costo de una compra es el de adquisición; el «costo» de una venta es el precio de venta. El COGS lo calcula FIFO, no el operador.
- Si se carga un stock inicial, el costo total es el del lote completo, no el unitario. El sistema divide.
- Editar o borrar un movimiento antiguo reescribe toda la cadena: el dashboard no «recuerda» el COGS anterior.
- Ajustar umbrales antes de interpretar alertas. Un almacén de movimiento lento (repuestos) necesita s , ℓ , m más holgados que un almacén de consumo diario.
- Elegir el período según la pregunta: 7 días para rupturas recientes, 90–365 para rotación estructural, Siempre para el retrato histórico.

- No borrar datos/inventario.json salvo para vaciar el laboratorio. Los backups permiten recuperar un estado de hace minutos.
- El .exe debe vivir en una carpeta escribible. Si datos/ queda dentro de un directorio de solo lectura, el guardado falla en silencio relativo (el renderer no tiene fs).
- Chart.js y SheetJS están vendorizados: no hace falta red para graficar ni para exportar.
- Mantenimiento: los umbrales por defecto, el tamaño de página (8) y el intervalo de backup (60 s) viven en script.js y main.js. No hay archivo .env.

9. Conclusiones

- FIFO como invariante: cada venta consume el lote más antiguo y deja un COGS auditable. El dashboard no es una estimación, es el recálculo de la bitácora.
- Rotación con dos relojes: las veces COGS / inventario promedio miden velocidad; los días sin movimiento miden silencio. El semáforo actúa sobre el segundo, que es el que dispara capital dormido.
- Portabilidad real: un .exe, un JSON y una carpeta de backups bastan para un laboratorio o un almacén sin servidor.
- Período como lente, no como tijera: el análisis se recorta, los lotes no. Eso evita el error de «reiniciar el FIFO cada mes».
- Exportación alineada: el Excel habla el mismo idioma que la pantalla (tres hojas, mismo rango, mismo método FIFO).
- Alineación pedagógica: el sistema materializa, en una herramienta operable, las definiciones de rotación, cobertura, COGS y PEPS del curso de Logística.

10. Referencias Técnicas

Ballou, R. H. Logística: administración de la cadena de suministro. Pearson. Capítulos de inventario y de rotación.

Chopra, S. y Meindl, P. Administración de la cadena de suministro. Pearson. Costos de mantenimiento y políticas de reposición.

Horngren, C. T., Datar, S. M. y Rajan, M. Contabilidad de costos. Pearson. Valoración de inventarios FIFO / PEPS y costo de lo vendido.

Silver, E. A., Pyke, D. F. y Thomas, D. J. Inventory and Production Management in Supply Chains. CRC Press. Días de inventario y cobertura.

Documentación de Electron (contextIsolation, contextBridge, app.isPackaged) y de electron-builder (target portable).

Chart.js 4. Documentation. <https://www.chartjs.org/docs/latest/>

SheetJS (xlsx). Community Edition. <https://docs.sheetjs.com/>

Anexo A. Instancia de demostración (equipos de red)

La semilla datos/inventario.json incluye ocho SKU de infraestructura (monitor industrial, servidor rack, switch, UPS, fibra, patch panel, router Wi-Fi 6, cámara PTZ) y 27 movimientos entre julio de 2024 y julio de 2026. Umbrales de la semilla: sana = 15, lenta = 50, muyLenta = 100 (más holgados que los de fábrica, acordes a un almacén de equipos). Se reproduce el FIFO del SKU «Monitor Industrial 4K» (p1):

Fecha	Tipo	Cant.	Unitario	Efecto FIFO
01/07/2024	Compra	50	\$ 400	Lote A: 50×400
15/09/2024	Venta	10	\$ 600	COGS = $10 \times 400 = 4\,000$; ingreso 6 000
10/05/2025	Venta	15	\$ 620	COGS = $15 \times 400 = 6\,000$; ingreso 9 300
20/01/2026	Venta	5	\$ 590	COGS = $5 \times 400 = 2\,000$; ingreso 2 950

Tabla 4. Replay FIFO del producto p1 (Monitor Industrial 4K).

Stock resultante: 20 unidades del lote A. Valor de inventario = $20 \times 400 = \$ 8\,000$. COGS acumulado de las tres ventas = \$ 12 000. Ingresos = \$ 18 250. Ganancia bruta = \$ 6 250. El precio de venta nunca entra a la cola de lotes: solo el costo de compra. Si el operador borra la compra inicial, el replay valorará las ventas al lastCost residual (regla de faltante), y el stock quedará en cero: la bitácora sigue siendo la autoridad, no la tarjeta de stock.

Anexo B. Esquema JSON de persistencia

Campo	Tipo	Rol
meta.version / createdAt	número / ISO	Versión de esquema y alta del archivo
config.metodoCosteo	«FIFO»	Constante de política; no editable en UI
config.umbrales.sana lenta muyLenta	enteros (días)	Cortes del semáforo
productos[].id	string	Clave interna (uid)
productos[].nombre	string	Único atributo de negocio del maestro
movimientos[].tipo	compra venta	Dirección del flujo
movimientos[].cantidad	entero ≥ 1	Unidades del documento
movimientos[].costoUnitario	número ≥ 0	Costo (compra) o precio (venta)
movimientos[].fecha	ISO-8601	Instante del replay
movimientos[]._cogs / _ingreso	número (derivado)	Se reescriben en cada compute()

Tabla 5. Esquema de datos/inventario.json.

No existe un arreglo de «lotes» persistido. Los lotes viven solo en memoria, dentro de `Engine.compute(db)`. Esa ausencia es una garantía: nadie puede «arreglar» el stock editando un lote a mano; hay que corregir la compra o la venta que lo originó.

— *Fin de la Memoria Descriptiva* —